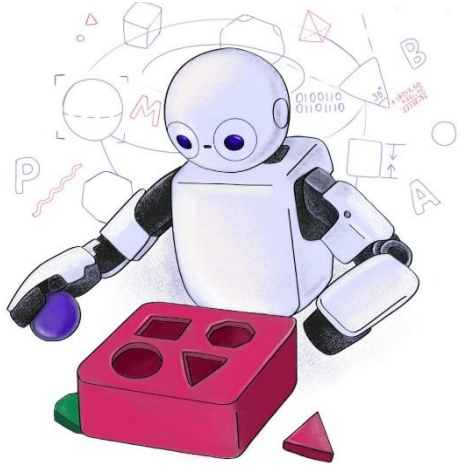
TP558 - Tópicos avançados em Machine Learning:

DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling

IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE 2023

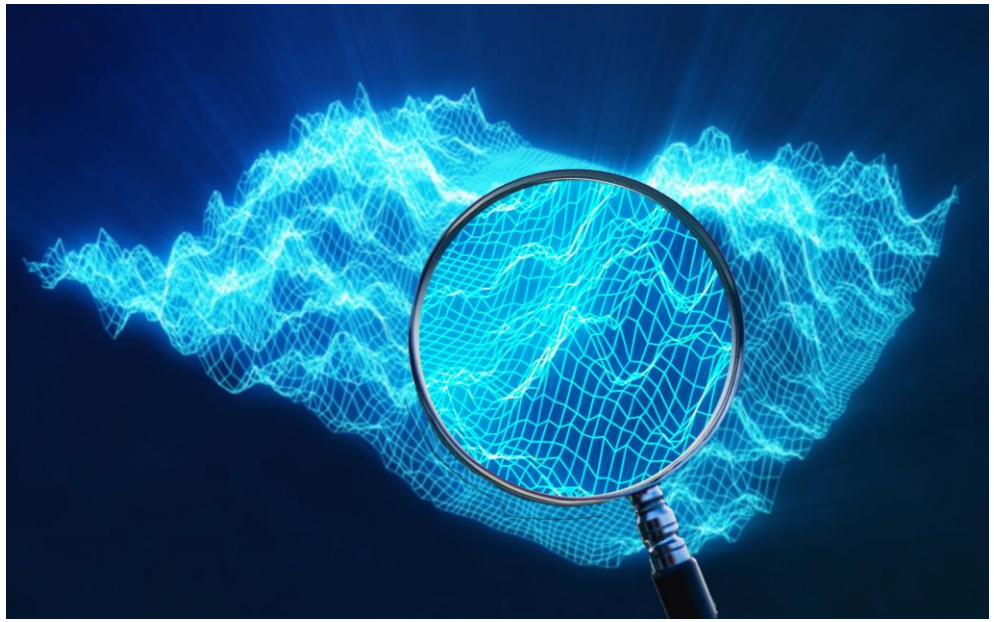


Inatel

Matheus Ferreira Silva
matheus.f@inatel.br

Contexto do Tema

- **NAS, Neural Architecture Search:**
- A área que busca **descobrir automaticamente arquiteturas neurais**, reduzindo a dependência de tentativa e erro.



Algorithm 1 Single-objective NAS algorithm.

Require: S008 data $\mathcal{D}_{008}$, S009 data $\mathcal{D}_{009}$, trials T, max epochs E
Ensure: best configuration Θ^* and architecture $\mathcal{A}^*$
split $\mathcal{D}_{008}$ into $\mathcal{D}_{\text{train}}$ (80%) and $\mathcal{D}_{\text{val}}$ (20%)
define family set $\mathcal{F}$ with candidate families:
CNN 1D, CNN 2D, Transformer-based, GNN, and CNN 1D+GNN
initialize an Optuna study with TPE and Hyperband
for trial $= 1$ **to** T **do**
Preprocess inputs:
fit a standard scaler on the training-set GPS data, then transform the training and validation GPS data
one-hot encode LiDAR $(20, 200, 10) \rightarrow (20, 200, 4)$ and reshape to $(4000, 4)$
tile GPS $(2,) \rightarrow (4000, 2)$ and concatenate to obtain $(4000, 6)$
Sample family $f \in \mathcal{F}$
sample family-specific hyperparameters from f
Sample optimization hyperparameters:
$\theta_{\text{opt}} \in \{\text{SGD}, \text{Adam}, \text{AdamW}, \text{Lion}, \text{RMSprop}\}$
$\theta_{\text{lr}} \sim \log \mathcal{U}(10^{-4}, 10^{-2})$, $\theta_{\text{bs}} \in \{32, 64, 128, 256\}$
build model $\mathcal{A}$ from f with the sampled hyperparameters
compile $\mathcal{A}$ with sparse categorical cross-entropy and $(\theta_{\text{opt}}, \theta_{\text{lr}})$
train on $\mathcal{D}_{\text{train}}$ with batch size θ_{bs} for up to E epochs with early stopping
report validation sparse categorical cross-entropy loss on $\mathcal{D}_{\text{val}}$ each epoch and prune if unpromising
store final validation loss $\mathcal{L}_{\text{val}}$
end for
select $(\Theta^*, \mathcal{A}^*) = \arg\min_{\Theta} \mathcal{L}_{\text{val}}$ over completed trials
retrain $\mathcal{A}^*$ with Θ^* under the S008 protocol and evaluate once on $\mathcal{D}_{009}$
return Θ^* and $\mathcal{A}^*$

DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Nov. 2023.

Contexto do Tema

- A motivação vem do fato de que arquiteturas projetadas por especialistas podem ter alto desempenho, mas exigem **muito ajuste empírico e conhecimento especializado**.

- **Neural Architecture Search, NAS**, busca automatizar o projeto de arquiteturas neurais, substituindo parte do ajuste manual de operações, conexões e larguras por um processo de busca orientado por desempenho e custo.

- Entre as linhas de NAS, uma das mais influentes é o **DARTS, Differentiable Architecture Search**.

DARTS, Differentiable Architecture Search

- O **DARTS** é um método de NAS que transforma a busca de arquitetura em um problema **diferenciável**, permitindo otimização por gradiente em vez de busca puramente discreta.

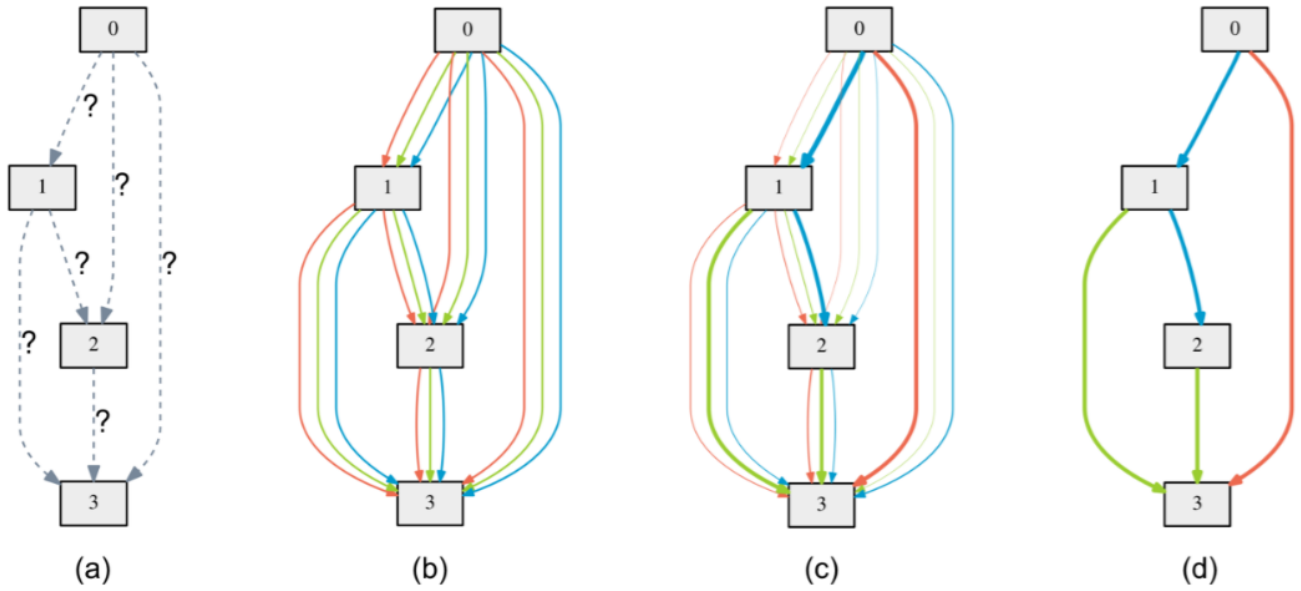


Figure 1: An overview of DARTS: (a) Operations on the edges are initially unknown. (b) Continuous relaxation of the search space by placing a mixture of candidate operations on each edge. (c) Joint optimization of the mixing probabilities and the network weights by solving a bilevel optimization problem. (d) Inducing the final architecture from the learned mixing probabilities.

H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable Architecture Search," in Proc. Int. Conf. Learn. Representations (ICLR), 2019

Introdução do artigo

- O paper parte da seguinte limitação:
  - NAS differentiable, como DARTS, funciona bem em classificação,
  - mas sua aplicação direta em detecção e segmentação ainda é limitada pelo custo do supernet.
- Há dois problemas centrais:
  - alto custo de memória e computação para buscar arquiteturas em tarefas complexas,
  - falta de uma ferramenta reutilizável e configurável para transformar diferentes search spaces em supernets treináveis.
- Métodos anteriores que reduzem custo por sampling ativam apenas parte do supernet a cada iteração.

Porém, isso introduz uma estrutura dinâmica durante a busca, o que pode dificultar a convergência, especialmente em tarefas complexas.

Introdução do artigo

A proposta do artigo é o **EasyNAS**, um framework de NAS para visão com três ideias principais:

- **convolution operator merging**, para mesclar convoluções candidatas em uma única convolução compartilhada,
- **dynamic channel refinement**, para buscar redução de canais sem computação redundante,
- **search space to supernet conversion tooling**, baseado em atomic search components, para construir supernets configuráveis para diferentes tarefas.

A meta do paper é:

1. reduzir o custo da busca,
2. manter melhor estabilidade de treinamento,
3. e tornar NAS mais reutilizável em classificação, detecção e segmentação.

Trabalhos relacionados

NAS para classificação

A literatura inclui abordagens por reinforcement learning, algoritmos evolutivos e métodos differentiable.

O DARTS se consolidou porque reduz o custo de busca ao tratar NAS como otimização bi-level em um supernet treinado por gradiente.

NAS para detecção

Os trabalhos anteriores podem ser organizados em três linhas:

- busca apenas do backbone,
- busca apenas da FPN,
- ou busca conjunta de backbone e FPN.

O problema é que muitos desses métodos ainda exigem alto custo de busca.

NAS para segmentação

Métodos como Auto-DeepLab e FasterSeg já mostraram que NAS pode ser aplicado à segmentação semântica.

Porém, o treinamento do supernet continua pesado em memória e computação.

Limitação comum destacada pelos autores: NAS já foi estendido para classificação, detecção e segmentação, mas ainda falta uma abordagem que seja ao mesmo tempo eficiente em custo e reutilizável entre diferentes tarefas de visão.

Trabalhos relacionados

DARTS

Trabalhos recentes tentam reduzir o custo do DARTS por três estratégias principais: sampling de sub-redes, ativação parcial de canais e pruning progressivo de operações fracas.

Vantagem dessas abordagens: reduzir o custo de memória do supernet, tornando a busca mais leve do que o DARTS original.

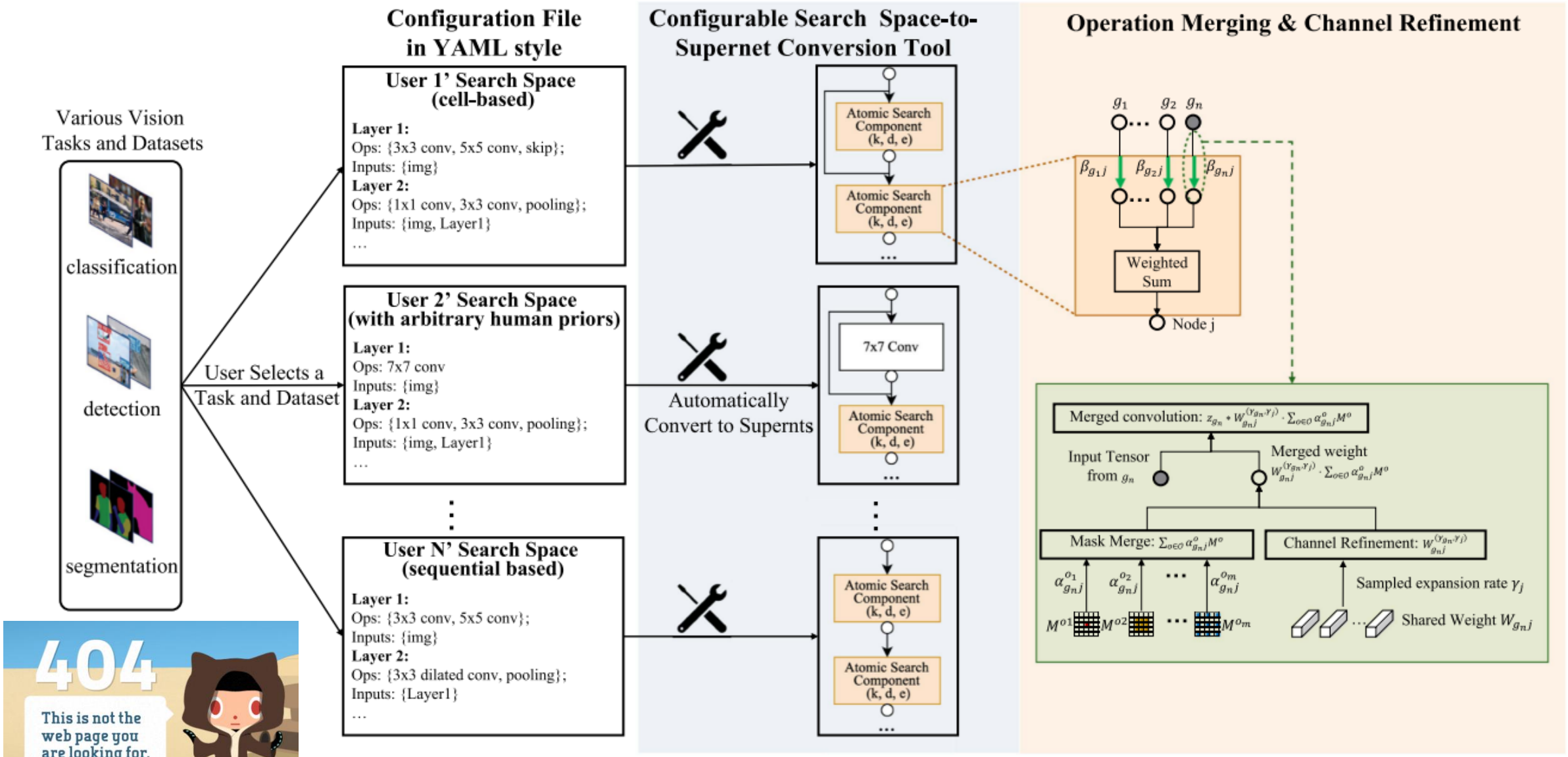
Limitação identificada: muitas dessas soluções tornam a estrutura do supernet dinâmica ao longo do treinamento, o que pode dificultar a convergência, especialmente em tarefas mais pesadas como detecção e segmentação.

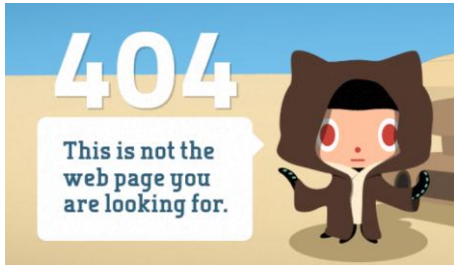
Lacuna que o artigo ocupa: o EasyNAS reduz o custo sem depender dessa dinâmica de sampling ou pruning como mecanismo principal. A proposta é manter um supernet mais estável, usando operator merging, dynamic channel refinement e uma ferramenta configurável de search space to supernet conversion.

Trabalhos relacionados

[2]Part of the work was published in the two conference works: [8] which applies the merge mechanism to classification, and [26] which designs a sophisticated search space for detection and applies the merge technique. The journal version aims to unify these methods under a single framework and makes the extensions and improvements: 1) to make it applicable to different vision tasks with user specified search space, we design and implement a reusable and configurable conversion tool from the given search space to supernet based on our proposed concept of atomic search component; 2) extensive additional experiments including i) classification on ImageNet as well as robustness analysis which is an emerging issue in recent NAS; ii) more detection results using four different detection heads beyond YOLO; iii) brand new results on segmentation which also verifies our method's general applicability; iv) the paper was rewritten with more comprehensive discussions on our approach and related works.

Modelo de Sistema





DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling, Nov. 2023.

Atomic search component

Atomic Search Component: bloco elementar de busca do EasyNAS. Ele pode ser visto como um pequeno grafo com um nó de saída j e um conjunto de entradas candidatas G_j. Em cada bloco, o método busca simultaneamente a **conexão**, a **operação** e a **largura em canais**.

Espaço de busca do bloco

$$G_j = \text{conjunto de entradas candidatas do nó } j$$

$$O_j = \text{conjunto de operações candidatas}$$

$$R_j = \text{conjunto de taxas candidatas de redução de canais}$$

Parâmetros arquiteturais

α^o_{gj}: importância da operação o na aresta $g \to j$

β_{gj}: importância da conexão $g \to j$

γ^r_j: importância da taxa de redução $r \in R_j$ no nó j

Normalização

$$\hat{\alpha}^o_{gj} = \frac{\exp(\alpha^o_{gj})}{\sum_{o' \in O_j} \exp(\alpha^{o'}_{gj})}$$

$$\hat{\beta}_{gj} = \frac{\exp(\beta_{gj})}{\sum_{g' \in G_j} \exp(\beta_{g'j})}$$

$$\hat{\gamma}^r_j = \frac{\exp(\gamma^r_j)}{\sum_{r' \in R_j} \exp(\gamma^{r'}_j)}$$

DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling, Nov. 2023.

Atomic search component

$$z_j = \sum_{g \in G_j} \hat{\beta}_{gj} \left[z_g * W_{gj}^{(r_g, r_j)} \cdot \left(\sum_{o \in O_j^p} \hat{\alpha}_{gj}^o M^o \right) \right]$$

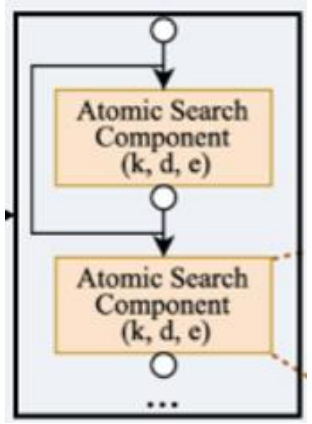
z_g: feature map de entrada do nó g

z_j: feature map de saída do nó j

$W_{gj}^{(r_g, r_j)}$: peso compartilhado da convolução entre g e j, refinado para os canais ativos

M^o: máscara associada à operação o

O_j^p: subconjunto das operações paramétricas, tipicamente convoluções



k: tamanho do kernel da convolução. Controla a escala espacial observada.
d: taxa de dilatação. Aumenta o receptive field da convolução.
e: taxa de redução dos canais de saída. Controla a largura e o custo do bloco.

DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling, Nov. 2023.

Atomic search component

$$z_j = \sum_{g \in G_j} \hat{\beta}_{gj} \left[z_g * W_{gj}^{(r_g, r_j)} \cdot \left(\sum_{o \in O_j^p} \hat{\alpha}_{gj}^o M^o \right) \right]$$

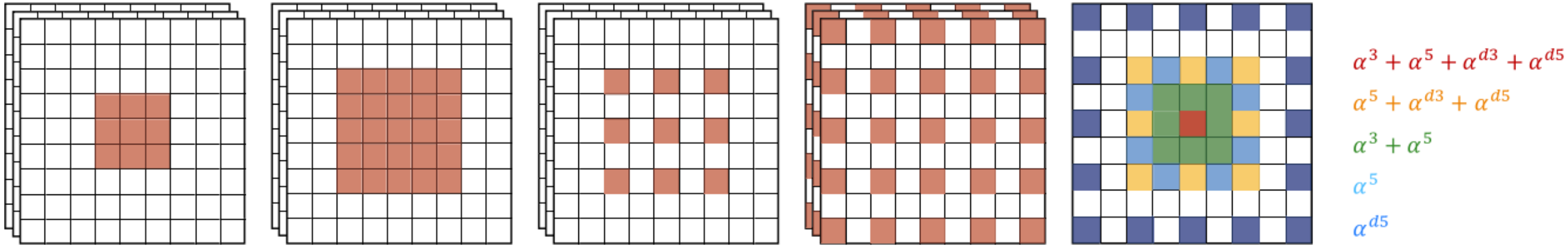
z_g: feature map de entrada do nó g

z_j: feature map de saída do nó j

$W_{gj}^{(r_g, r_j)}$: peso compartilhado da convolução entre g e j, refinado para os canais ativos

M^o: máscara associada à operação o

O_j^p: subconjunto das operações paramétricas, tipicamente convoluções



(a) Mask for 3×3 convolution (b) Mask for 5×5 convolution (c) Mask for 3×3 dilated convolution (d) Mask for 5×5 dilated convolution (e) mask matrix $\tilde{M}$

Fig. 2. (a)-(d) denote the equivalent mask M^o indicating the receptive field for different convolutions, where the red block denotes non-zero weights. (e) is the weighted average mask $\tilde{M}$ according to the architecture parameters α. Different colors refer to the corresponding coefficients.

DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling, Nov. 2023.

Pesos das convoluções

Dynamic channel refinement: se o nó j usa apenas $r_j \cdot C_j$ canais, então os pesos das convoluções adjacentes também são refinados:

- $g \rightarrow j$: produz apenas os canais ativos de j
- $j \rightarrow p$: recebe apenas os canais ativos de j

W: pesos treináveis da convolução em uma aresta do supernet.
No EasyNAS, W é **compartilhado** entre várias operações candidatas.

Operator merging: a convolução efetiva é obtida aplicando a máscara ponderada ao peso compartilhado:

$$\tilde{M} = \sum_{o \in O_j^p} \hat{\alpha}_{gj}^o M^o$$

$$\tilde{W}_{gj} = \tilde{M} \odot W_{gj}$$

r_j: taxa de redução de canais amostrada para o nó j

$$\text{canais ativos em } j = r_j \cdot C_j$$

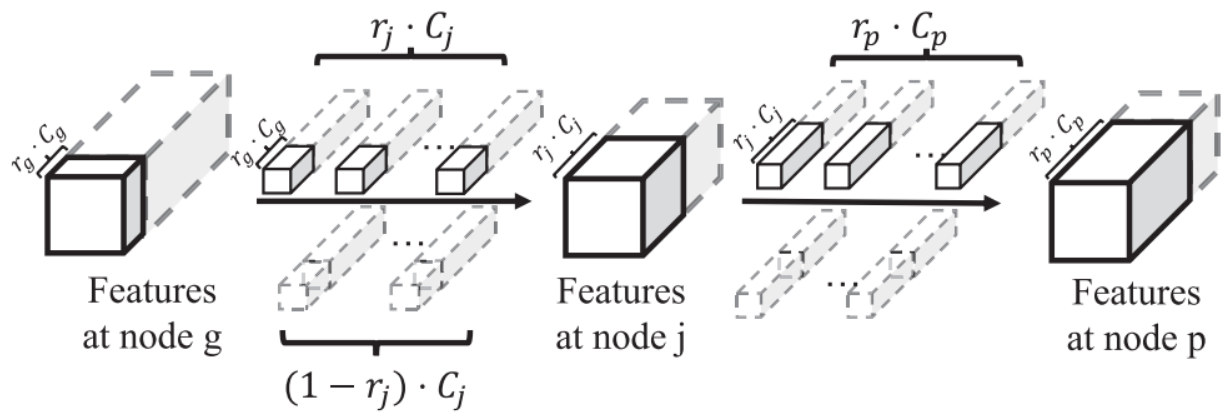


Fig. 5. Refinement for convolution weights W on adjacent layers in our dynamic channel refinement technique. r indicates the sampled reduction rate. Dotted blocks in light gray indicate the inactivated channels.

DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling, Nov. 2023.

Algoritmo de treinamento

1. Construção do supernet:

o search space definido pelo usuário é convertido em um supernet composto por **atomic search components**.

2. Inicialização:

são inicializados:

- W: pesos compartilhados da rede
- α: pesos das operações
- β: pesos das conexões
- γ: pesos das taxas de redução de canais

3. Em cada iteração de busca:

a) normalizar os parâmetros arquiteturais de operação, conexão e redução de canais

b) amostrar as taxas de redução de canais

c) refinar os pesos compartilhados das convoluções adjacentes para os canais ativos

d) formar a operação efetiva por operator merging

e) executar o forward do supernet

f) atualizar alternadamente:

- pesos da rede com dados de treino
- parâmetros arquiteturais com dados de validação

Algorithm 1: EasyNAS for General Architecture Search (Applicable to Classification, Detection, Segmentation, etc.).

Input:
1) A supernet constructed by stacking atomic search component with search space $\{\mathcal{G}, \mathcal{O}, \mathcal{R}\}$;
2) The initialized sharing weights $\mathbf{W}$ to merge convolutions on each edge;
3) The mask matrix for each operation $\mathbf{M}^o, o \in O$;

Hyperparameters:
1) The maximal epoch T to train the supernet weights and architecture parameters;

Output:
1) The optimal architecture parameters α, β, γ;

1: **for** $t = 1$ **to** T **do**
2: Get normalized architecture parameters $\{\hat{\alpha}, \hat{\beta}, \hat{\gamma}\}$ through softmax;
3: Sample reduction rate based on $\hat{\gamma}$ for each node through (8);
4: Apply dynamic channel refinement to convolution weights based on Fig. 5;
5: Supernet forward by computing the atomic search component in turn by (10);
6: Update architecture parameters $\{\alpha, \beta, \gamma\}$ and supernet weights W alternately through SGD.
7: **end for**
8: **return** the optimal architecture parameters $\{\alpha, \beta, \gamma\}$

DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling, Nov. 2023.

Célula de Busca para Classificação

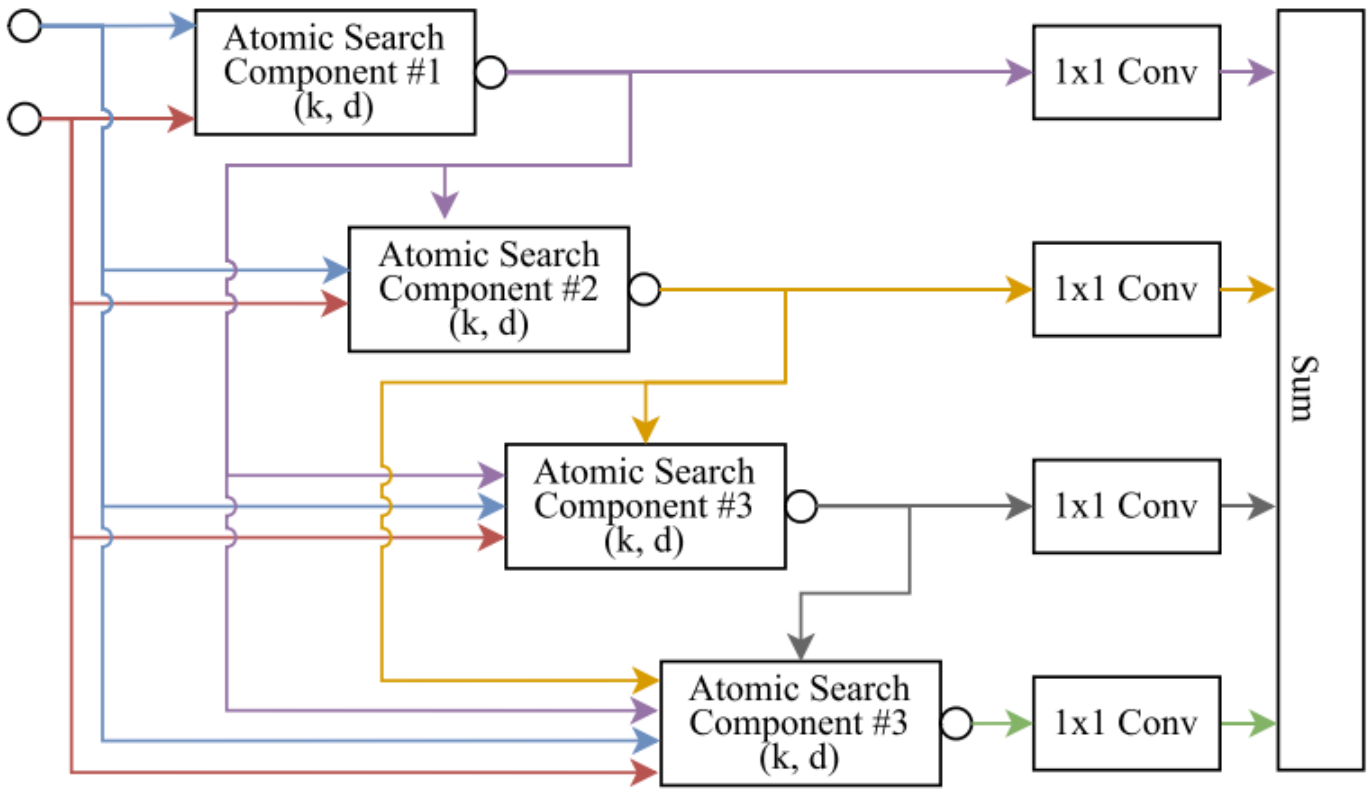


Fig. 6. Architecture of the cell for classification task based on the search space of DARTS [7]. The architecture of the atomic search component is illustrated in Fig. 1. Lines in the same color are from the same source node. The candidate operation set $\mathcal{O}$ contains eight operations in each atomic search component.

- A célula tem **nós de entrada** e **nós intermediários**
- Cada nó intermediário é implementado como um **Atomic Search Component**
- Cada componente recebe **várias entradas candidatas dos nós anteriores**
- Em cada conexão, o método escolhe a operação mais adequada dentro do conjunto candidato

Para classificação, o EasyNAS consegue reproduzir a lógica de uma célula tipo DARTS, mas substituindo os blocos tradicionais pelos seus **atomic search components**.

DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling, Nov. 2023.

Supernet para Detecção de Objetos

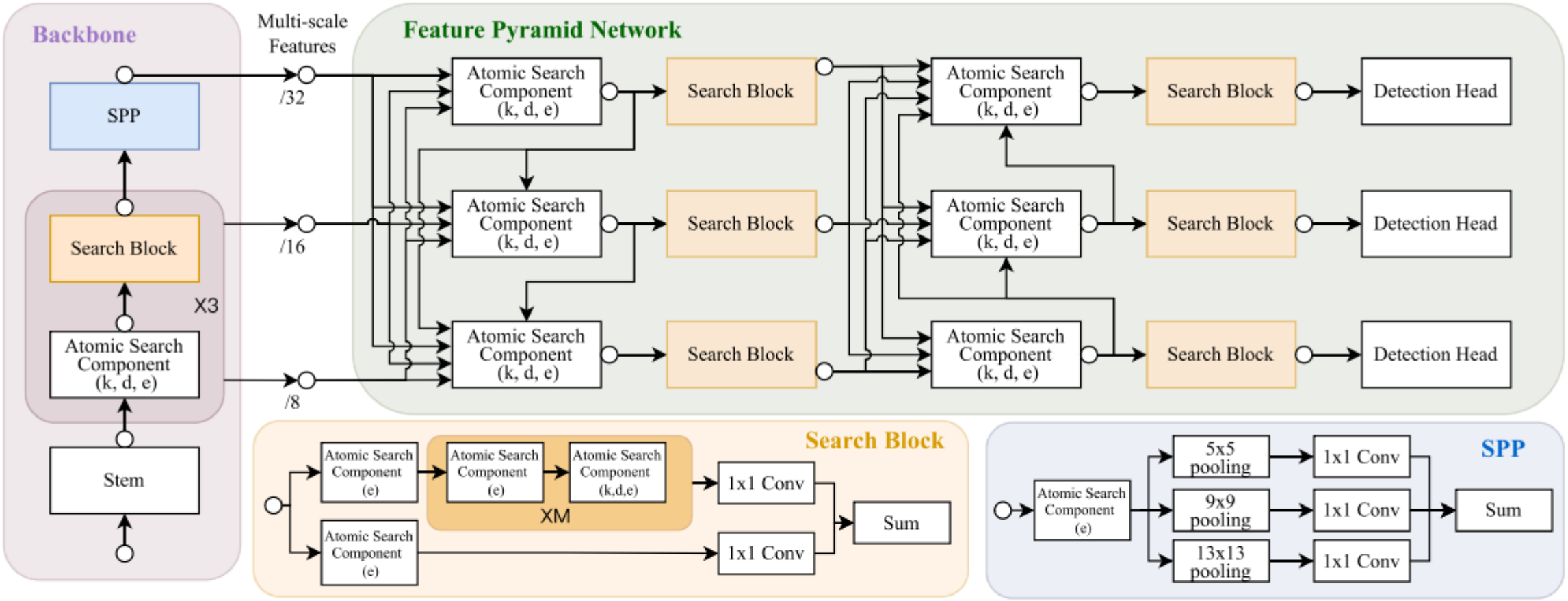


Fig. 7. Architecture of supernet, containing all candidate operations and connections in the search space. The architecture of the atomic search component is given in Fig. 1. In the backbone, search block, and SPP module, parentheses under 'Atomic Search Component' indicate hyper-parameters to search: kernel size k, dilation ratio d, and reduction rate of output channels e.

DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling, Nov. 2023.

Supernet para Segmentação Semântica

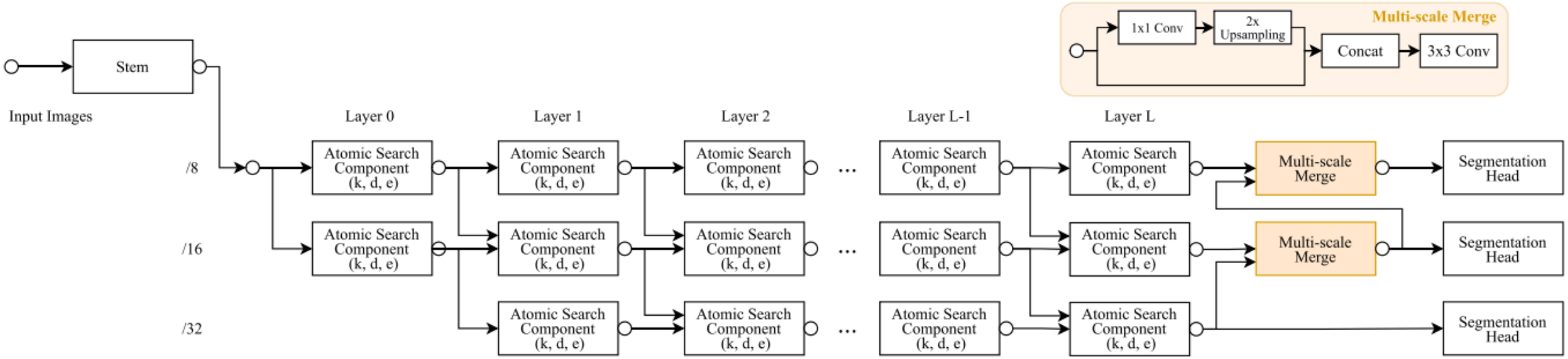


Fig. 8. Architecture of supernet based on the search space of FasterSeg [17]. The atomic search component is illustrated in Fig. 1. The parentheses under 'Atomic Search Component' indicate hyper-parameters to search: kernel size k, dilation ratio d, and reduction rate of output channels e. The 'Multi-scale Merge' block has the same architecture as that in FasterSeg.

DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling, Nov. 2023.

Resultados

Objetivo da avaliação: verificar a eficácia do EasyNAS em três tarefas clássicas de visão:

- **classificação**, com EasyNAS(cls)
- **detecção de objetos**, com EasyNAS(det)
- **segmentação semântica**, com EasyNAS(seg)

Estratégia geral de busca:

- para cada tarefa, o conjunto de treino foi dividido em duas partes
- uma parte foi usada para atualizar os **pesos da rede**
- a outra foi usada para atualizar os **parâmetros arquiteturais**
- a busca foi feita por **50 épocas**, com otimização alternada entre pesos e arquitetura
- ao final, a arquitetura foi discretizada preservando:
  - conexões
  - operações
  - taxas de redução de canais

Plataforma experimental:

- implementação em **PyTorch 1.7.1**
- classificação em **NVIDIA 1080 Ti**
- detecção e segmentação em **NVIDIA V100**

Classificação:

- busca em **CIFAR-10**
- avaliação também em **ImageNet**
- no search space tipo DARTS, o supernet de busca tem **8 células**
- a arquitetura final é treinada do zero por **600 épocas**
- na avaliação final, a rede usa **20 células**
- também há avaliação no **NAS-Bench-201**, onde a arquitetura encontrada é comparada diretamente com o benchmark

Detecção:

- busca feita em **MS-COCO 2017**
- o método busca conjuntamente **backbone + FPN**
- os modelos descobertos são treinados do zero por **300 épocas**
- os hiperparâmetros seguem o **YOLOv5**
- além do COCO, as arquiteturas descobertas são transferidas para:
  - **DOTA-v1.0**, em detecção rotacionada
  - outros detection heads, como **Faster R-CNN, RetinaNet, FCOS e Mask R-CNN**

Segmentação:

- avaliação em **Cityscapes**
- os pesos da rede são pré-treinados por **20 épocas** com arquitetura fixa
- depois a busca é feita por **30 épocas**
- a arquitetura final descoberta é treinada do zero por **600 épocas**

Resultados

TABLE II
GPU MEMORY COST OF EASYNAS AND DARTS ON THREE TASKS

Method	Classification	Object Detection	Segmentation
DARTS [7]	9.9 G (↓0%)	16.5 G (↓0%)	12.4 G (↓0%)
GDAS [21]	3.1 G (↓68%)	10.6 G (↓36%)	7.1 G (↓43%)
EasyNAS	4.6 G (↓54%)	11.0 G (↓33%)	7.3 G (↓41%)

The search spaces for the three tasks are described in Section IV-A2, IV-B2, and IV-C2. The relative memory reduction against darts is in the bracket.

TABLE I
SEARCH COST OF PADDING ZERO TECHNIQUE IN [27] AND OUR DYNAMIC CHANNEL REFINEMENT STRATEGY

Method	Baseline	Zero padding ([27])	Dynamic Channel Refinement (ours)
Cost (GPU-hours)	36.6	41.9	33.1

We search architectures for detection tasks on MS-COCO. The supernet is built based on EasyNas(det)-YOLOv5s (see fig. 7). Each supernet is trained for 50 epochs. 'Baseline' represents not searching for channel numbers.

DIY Your EasyNAS for Vision: Convolution Operation Merging, Map Channel Reducing, and Search Space to Supernet Conversion Tooling, Nov. 2023.

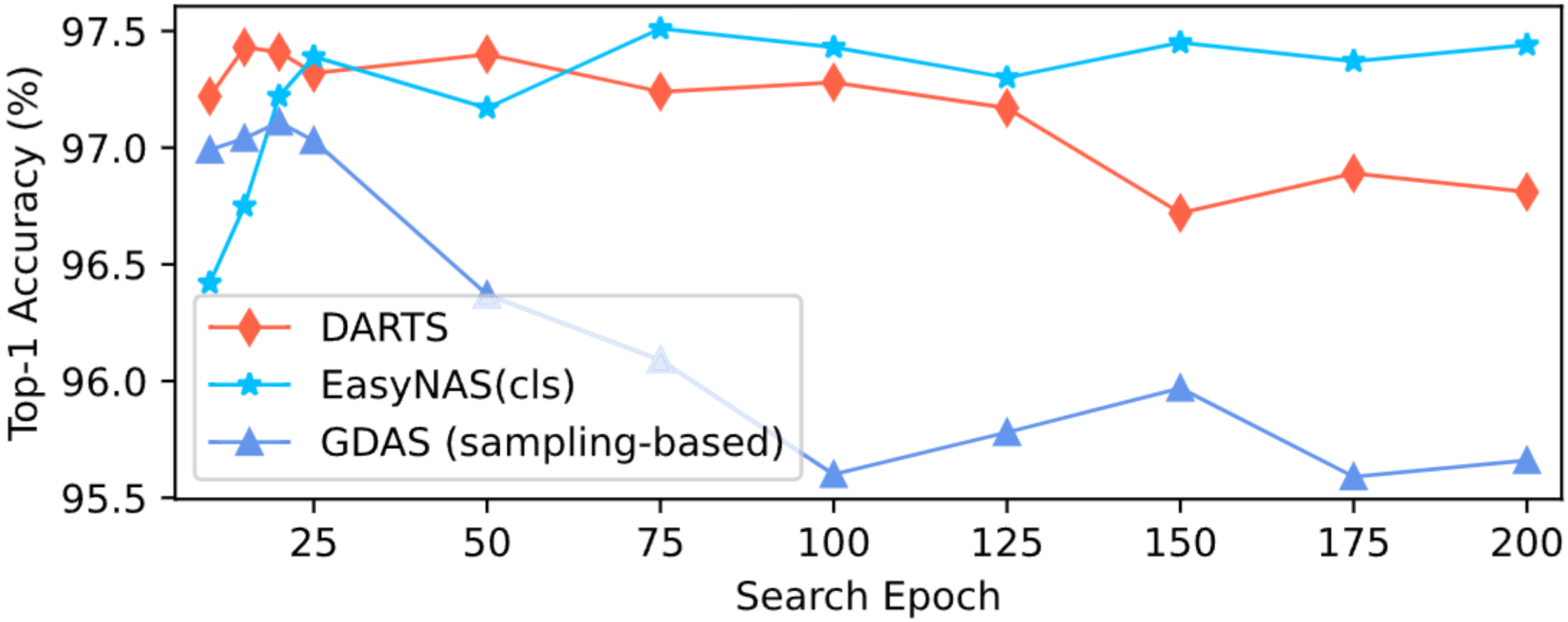


Fig. 4. Trends of ultimate accuracy of the discovered architecture in the search process of DARTS, EasyNAS(cls), and GDAS which is a popular sampling-based differentiable NAS method. The top-1 accuracy of each network is obtained by training for 600 epochs from scratch.

TABLE III
COMPARISON WITH STATE-OF-THE-ART IMAGE CLASSIFIERS ON CIFAR10
(LOWER ERROR RATE IS BETTER)

Classification Method	Top-1 Err. (%) ↓	Params (M) ↓	Cost (GPU-days) ↓
NASNet-A [3]	2.65	3.3	1800
AmoebaNet-B [6]	2.55±0.05	2.8	3150
Hierarchical Evo [5]	3.75±0.12	15.7	300
PNAS [29]	3.41±0.09	3.2	225
ENAS [4]	2.89	4.6	0.5
DARTS (1st order) [7]	3.00±0.14	3.3	0.4
DARTS (2nd order) [7]	2.76±0.09	3.4	1.0
SNAS (moderate) [60]	2.85±0.02	2.8	1.5
ProxylessNAS [61]	2.08	5.7	4.0
P-DARTS [52]	2.50	3.4	0.3
PC-DARTS (1st order) [20]	2.57±0.07	3.6	0.1
BayseNAS [62]	2.81±0.04	3.4	0.2
EasyNAS(cls) (1st order)	2.65±0.08	2.9	0.2
EasyNAS(cls) (2nd order)	2.55±0.07	3.0	0.6

Similar to darts, the search cost for EasyNAS does not include the selection or final evaluation cost. The performance values of compared methods are excerpted from the raw paper. We evaluate EasyNAS(cls) three times under different random seeds and report the averaged accuracy.

TABLE IV
PERFORMANCE ON IMAGENET IN DARTS'S SEARCH SPACE [7] BY TWO ARCHITECTURES

Classification Method	FLOPs (M) ↓	Params (M) ↓	Top-1 Err. (%) ↓	Cost (GPU-days) ↓
AmoebaNet-A [6]	555	5.1	25.5	3150
NASNet-A [63]	564	5.3	26.0	1800
PNAS [29]	588	5.1	25.8	225
DARTS (2nd) [7]	574	4.7	26.7	1.0
P-DARTS [52]	557	4.9	24.4	0.3
PC-DARTS [20]	586	5.3	25.1	0.1
SNAS [60]	522	4.3	27.3	1.5
GDAS [21]	581	5.3	26.0	0.2
ProxylessNAS† [61]	465	7.1	24.9	8.3
EasyNAS(cls)-singleSep	485	4.3	26.0	0.2
EasyNAS(cls)	696	6.0	23.8	0.2

'Singlesep' denotes that the separable convolution in the search space is single, similar to ENAS [4]; Otherwise, the separable convolution in the search space is double, similar to darts [7].

TABLE VII
COMPARISON WITH PRIOR WORKS ON THE COCO TEST-DEV

Object Detection Method	Resolution	FPS	#Params (M)	mAP (%)	AP_{50} (%)	AP_{75} (%)	AP_S (%)	AP_M (%)	AP_L (%)	Search Cost
YOLOv4 [30]	416	96	-	41.2	62.8	44.3	20.4	44.4	56.0	-
YOLOv5s† [66]	640	113	7.3	36.9	56.0	40.0	19.9	41.1	46.0	-
YOLOv7-tiny [67]	640	182	6.2	38.6	56.8	41.5	21.2	42.7	52.3	-
EfficientDet-D0 [39]	512	98	3.9	33.8	52.2	35.8	12.0	38.3	51.2	-
NAS-FPN [23]	640	24	60.3	39.9	-	-	-	-	-	333*
NAS-FCOS@128 [25]	1333×800	-	27.8	37.9	-	-	-	-	-	28
SpineNet-49S [68]	640	-	11.9	39.5	59.3	43.1	20.9	42.2	54.3	-
SM-NAS:E2 [37]	800×600	25	-	40.0	58.2	43.4	21.1	42.4	51.7	187
EasyNAS(det)-YOLOv5s (ours)	**640**	**120**	**9.1**	**40.1**	**58.7**	**43.5**	**21.7**	**43.8**	**50.5**	**1.4**
EasyNAS(det)-YOLOv7-tiny (ours)	**640**	**159**	**7.0**	**38.9**	**57.2**	**41.2**	**19.9**	**42.0**	**54.8**	**1.9**

TABLE XI
COMPARISON ON THE TEST SET OF ORIENTED BOUNDING BOXES (OBB) TASK IN DOTA-V1.0 FOR ROTATION DETECTION

Rotation Detection Method	PL	BD	BR	GTF	SV	LV	SH	TC	BC	ST	SBF	RA	HA	SP	HC	mAP_{50}
ReDet(ReR50) [32]*	88.79	82.64	53.97	74.00	78.13	84.06	88.04	**90.89**	87.78	85.75	61.76	60.39	75.96	68.07	63.39	76.25
CenterMap(R101) [76]*	89.83	84.41	**54.60**	70.25	77.66	78.32	87.19	90.66	84.89	85.27	56.46	69.23	74.13	71.56	66.06	76.03
CSL(R152) [36]* †	**90.13**	84.43	54.57	68.13	77.32	72.98	85.94	90.74	85.95	86.36	63.42	65.82	74.06	73.67	70.08	76.24
O^2-DNet(H104) [77]	89.31	82.14	47.33	61.21	71.32	74.03	78.62	90.76	82.23	81.36	60.93	60.17	58.21	66.98	61.03	71.04
DAL(R101) [78]	88.61	79.69	46.27	70.37	65.89	76.10	78.53	90.84	79.98	78.41	58.71	62.02	69.23	71.32	60.65	71.78
P-RSDet(R101) [79]	88.58	77.83	50.44	69.29	71.10	75.79	78.66	90.88	80.10	81.71	57.92	63.03	66.30	69.77	63.13	72.30
BBAVectors(R101) [80]	88.35	79.96	50.69	62.18	78.43	78.98	87.94	90.85	83.58	84.35	54.13	60.24	65.22	64.28	55.70	72.32
DRN(H104) [81]†	89.71	82.34	47.22	64.10	76.22	74.43	85.84	90.57	86.18	84.89	57.65	61.93	69.30	69.63	58.48	73.23
DCL(R152) [35]†	89.10	84.13	50.15	73.57	71.48	58.13	78.00	**90.89**	86.64	86.78	67.97	67.25	65.63	74.06	67.05	74.06
PolarDet(R101) [82]†	89.65	**87.07**	48.14	70.97	78.53	80.34	87.45	90.76	85.63	86.87	61.64	**70.32**	71.92	73.09	67.15	76.64
GWD(R152) [33]†	86.96	83.88	54.36	**77.53**	74.41	68.48	80.34	86.62	83.41	85.55	**73.47**	67.77	72.57	75.76	73.40	76.30
CSL(YOLOv5-s) [83]	88.93	76.07	50.97	59.61	81.38	84.00	88.19	90.81	80.52	87.63	47.38	62.85	68.27	80.63	62.74	74.00
CSL(**EasyNAS(det)-s**)	88.64	84.78	51.02	62.04	81.30	84.02	88.23	90.82	85.79	87.36	52.20	65.68	73.72	74.33	65.93	75.72
CSL(YOLOv5-m) [83]	88.36	85.24	53.86	62.35	81.19	84.72	**88.57**	90.77	80.98	**88.09**	53.92	61.90	**75.99**	80.43	**68.59**	76.33
CSL(**EasyNAS(det)-m**)	89.04	86.61	53.83	63.05	**81.51**	**85.12**	88.46	90.77	**88.21**	87.93	56.04	60.96	75.96	**82.13**	66.18	**77.05**

R152 denotes ResNet-152 [1] and H-104 denotes hourglass-104 [74]. †: models are trained with multi-scale training techniques, while others are not. *: two-stage detection framework, while others belong to a one-stage framework. Note CSL refers to the rotation detection loss which can be embodied by different backbones.

TABLE XII
EVALUATION OF SEMANTIC SEGMENTATION MODELS ON CITYSCAPE

Segmentation Method	mIoU (%)	FLOPs (G)	FPS	Search Cost (GPU-days)	Resolution
FCN-8S [40]	65.3	-	4.4	-	512x1024
PSPNet [85]	81.2	-	0.8	-	713x713
DeepLabV3 [41]†	81.3	-	1.3	-	769x769
Fast-SCNN [86]	68.0	-	123.5	-	1024x2048
AutoDeeplab [18]	80.3	695	-	3	1024x2048
CAS [87]	70.5	-	108	-	768x1536
GAS [88]	71.8	-	108	-	768x1536
SparseMask [89]	68.6	36.4	110	4.2	512x512
FasterSeg [17]	71.5	28.2	164	1.5	1024x2048
EasyNAS(seg)	72.6	28.2	173	0.7	1024x2048

Top: the performance of manually-designed architectures; Bottom: The performance of architectures searched by NAS. The search space is designed based on fasterseg [17] for fair comparison and differs from AutoDeeplab [18], as AutoDeeplab's codes for PyTorch platform are unavailable. †: extra coarse annotation data in cityscape in cityscape is utilized for training.

Conclusões

- Proposta: EasyNAS, um framework de NAS differentiable para visão com foco em eficiência computacional.
- O método consegue reduzir o custo do supernet sem depender de uma estrutura dinâmica por sampling como mecanismo principal.
- Os experimentos mostram que o EasyNAS mantém boa convergência, melhora a robustez da busca e alcança resultados competitivos em:
  - classificação
  - detecção
  - detecção rotacionada
  - segmentação semântica

Trabalhos Futuros

- Expandir o framework para outros search spaces e arquiteturas mais diversas.
- Investigar a aplicação do método em mais tarefas de visão computacional.
- Explorar novas extensões do framework para além de convoluções e estruturas já estudadas.
- Continuar desenvolvendo a ideia de um pipeline de NAS mais geral, reutilizável e fácil de aplicar.

Limitações

- O método ainda depende de um supernet differentiable, então não elimina totalmente os problemas clássicos de one-shot NAS.
- A generalidade do framework vale sobretudo para arquiteturas convolucionais de visão.
- O operator merging depende de operações que possam ser representadas por kernels compartilhados e máscaras.
- A busca por redução de canais aumenta a flexibilidade, mas também adiciona complexidade à otimização.

- O paper foca mais em desempenho final e custo do que em análise detalhada de:
  - sensibilidade a hiperparâmetros
  - variabilidade entre execuções
  - contribuição isolada de cada componente

- Ainda há trade-offs em aberto entre:
  - espaço de busca
  - custo de busca
  - qualidade final da arquitetura

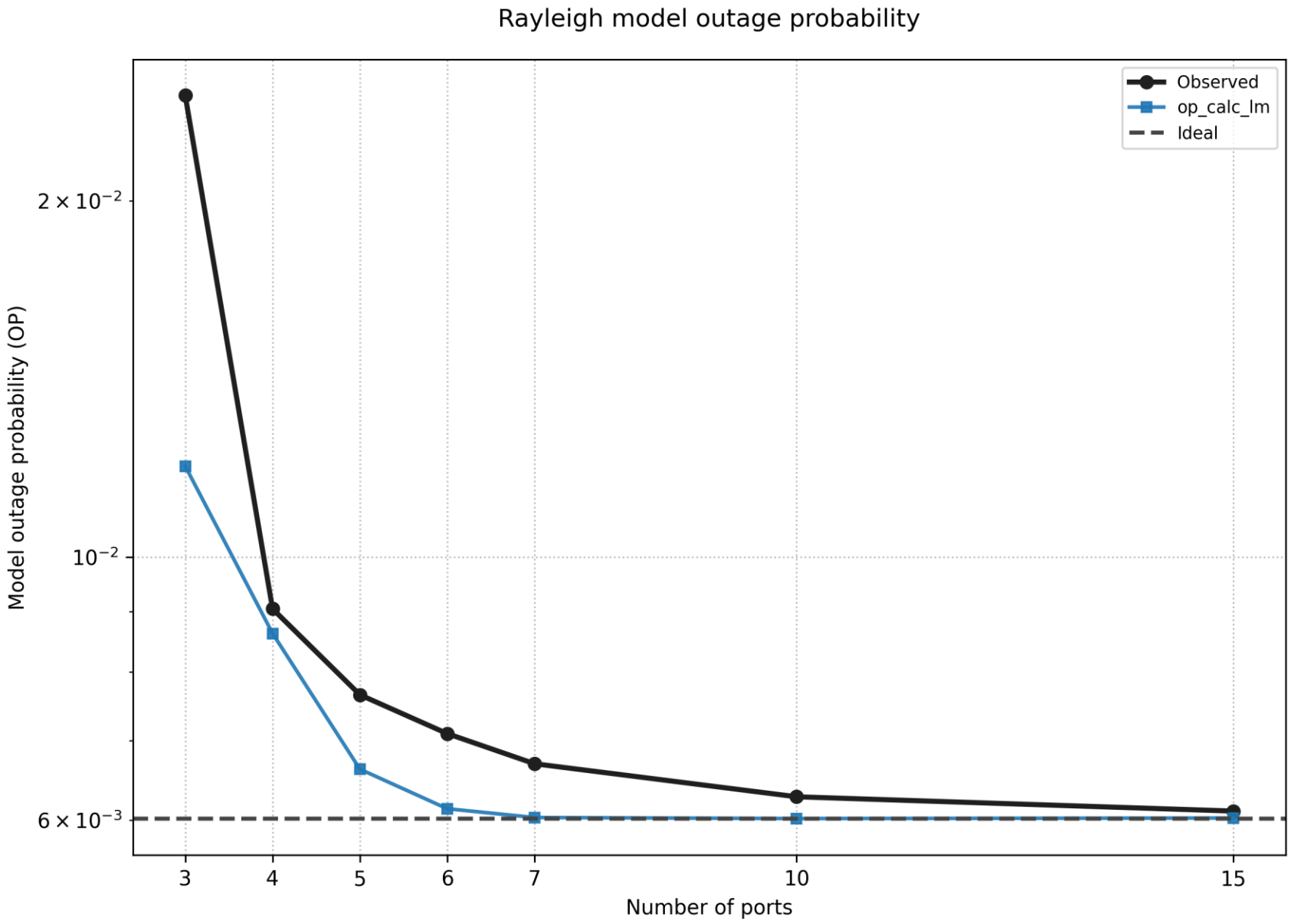
Minhas implementações

TABLE IV

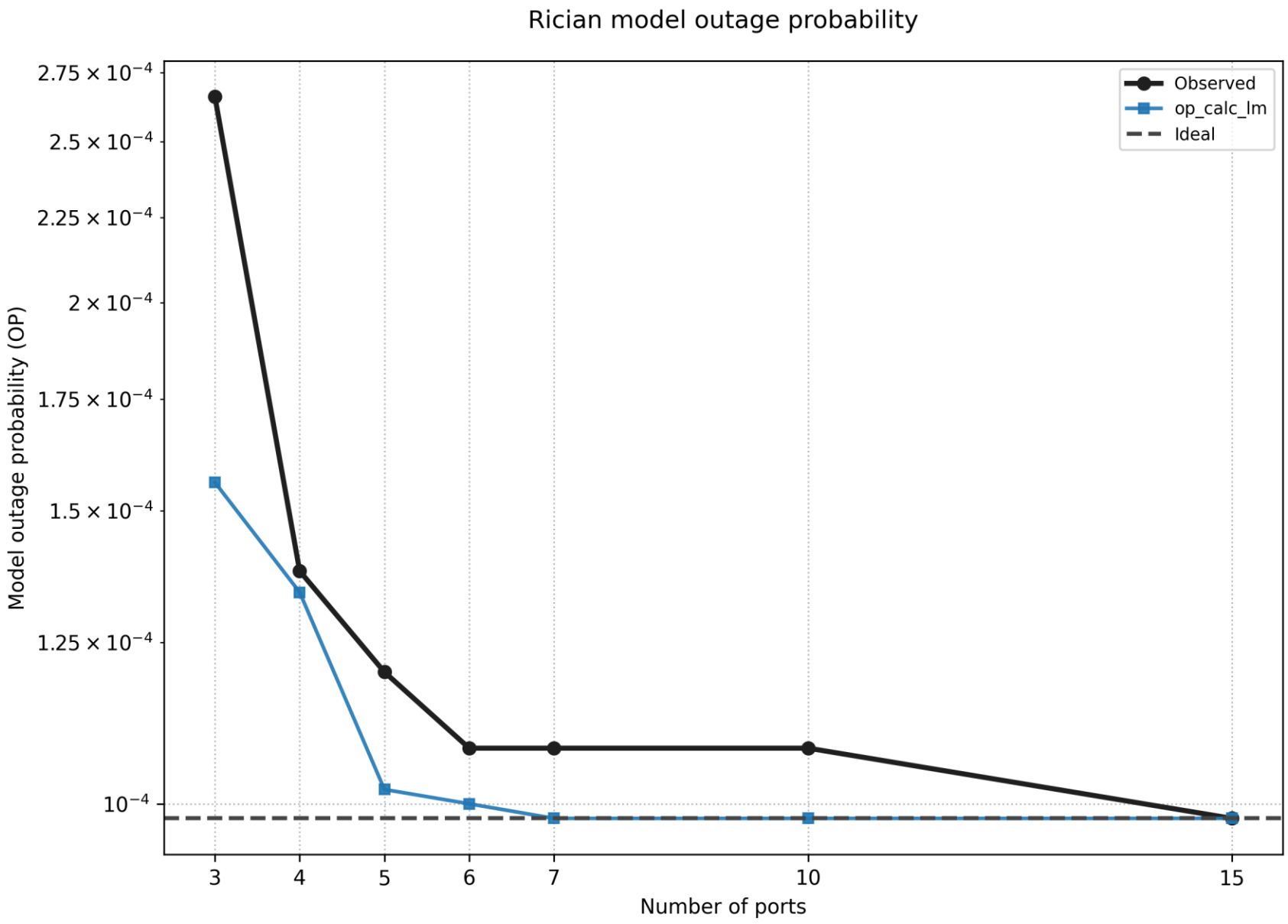
COMPARISON WITH PRIOR WORKS ON S009 UNDER JOINT EVALUATION.

Work	# Params	FLOPs	t_{inf}	Top-1	Top-5	Top-10	Top-50
[2]	31k	4.55M	1.1 ms	59.50 %	87.00 %	92.20 %	–
[6]	2.60M	226.1M	1.7 ms	56.22 %	85.53 %	91.11 %	98.60 %
[7]	723k	760.9M	2.1 ms	57.53 %	87.96 %	93.40 %	99.07 %
[9]	319k	70.2M	2.6 ms	59.87 %	86.13 %	92.23 %	–
Proposed	230k	321.9M	2.3 ms	61.16 %	86.71 %	92.08 %	98.32 %
Proposed-Ensemble	–	–	3.6 s	62.59 %	87.57 %	93.68 %	99.46 %
Proposed-Distilled (TPE)	230k	321.9M	2.3 ms	61.59 %	87.59 %	93.43 %	99.41 %
Proposed-Distilled (MOTPE)	157k	40.7M	0.49 ms	61.23 %	86.98 %	92.30 %	98.78 %

Minhas implementações



Minhas implementações



Perguntas?

Quiz

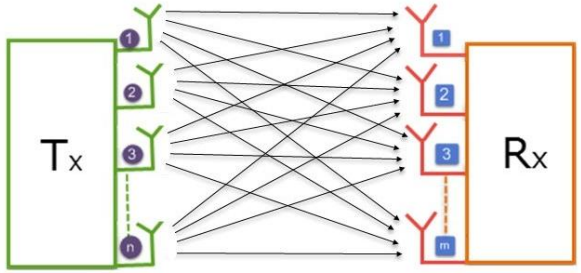
https://forms.gle/BVNU8KLtxvJSaC5V9

Referências

[1] Y. Huang et al., “Liquid antennas: Past, present and future,” IEEE OpenJ. Antennas Propag., vol. 2, pp. 473–487, 2021.
[2] C. Borda-Fortuny et al., “Low-cost 3D-printed coupling-fed frequencyagile fluidic monopole antenna system,” IEEE Access, vol. 7,pp. 95058–95064, 2019.
[3] Y. Shen et al., “Radiation pattern diversified double-fluid-channelsurface-wave antenna for mobile communications,” in Proc. IEEE-APSTopical Conf. Antennas Propag. Wireless Commun. (APWC), 2022,pp. 85–88.
[4] H. Wang et al., “Continuous electrowetting surface-wave fluid antennafor mobile communications,” in Proc. IEEE Region 10 Conf. (TENCON),2022, pp. 1–4.
[5] B. A. Cetiner et al., “Multifunctional reconfigurable MEMS integratedantennas for adaptive MIMO systems,” IEEE Commun. Mag., vol. 42,no. 12, pp. 62–70, Dec. 2004.
[6] A. Grau Besoli and F. De Flaviis, “A multifunctional reconfigurable pixeled antenna using MEMS technology on printed circuitboard,” IEEE Trans. Antennas Propag., vol. 59, no. 12, pp. 4413–4424,Dec. 2011.
[7] L. Jing et al., “Compact pattern reconfigurable pixel antenna withdiagonal pixel connections,” IEEE Trans. Antennas Propag., vol. 70,no. 10, pp. 8951–8961, Oct. 2022.
[8] K.-K. Wong et al., “Fluid antenna systems,” IEEE Trans. WirelessCommun., vol. 20, no. 3, pp. 1950–1962, Mar. 2021.
[9] K. K. Wong and K. F. Tong, “Fluid antenna multiple access,”IEEE Trans. Wireless Commun., vol. 21, no. 7, pp. 4801–4815,Jul. 2022.
[10] K.-K. Wong et al., “Slow fluid antenna multiple access,” IEEE Trans.Commun., vol. 71, no. 5, pp. 2831–2846, May 2023.

Obrigado!

Fundamentação teórica



- **MIMO** é um sistema com múltiplas antenas no transmissor e/ou receptor. Permite maior eficiência espectral, diversidade e melhor exploração do canal sem fio.
- **CSI (Channel State Information):** Informação que descreve o canal entre transmissor e receptor. É usada para modelar o sistema e auxiliar a predição dos sinais não observados.
- **RZF (Regularized Zero-Forcing):** Detector linear usado para recuperar o sinal desejado e reduzir interferência. A regularização torna a inversão mais estável na presença de ruído e correlação.
- **BER e NMSE:** BER mede erro na detecção dos bits. NMSE mede o erro na predição dos sinais estimados.